CAM 5 Database Application Report

Grand Luxe Hotel Management System

Full-Stack Relational Database Web Application • MySQL 8.0 & Python Flask

Student Degree: B.Tech Computer Science & Engineering | **Academic Year**: 2026
Evaluation Target: Laboratory Practical Assessment & Viva Examination

Executive Summary: This report presents the architectural design, schema implementation, query formulations, testing outcomes, and administrative dashboard for the CAM 5 Hotel Management Database System. The application establishes a normalized 10-entity MySQL 8.0 schema with referential integrity, automated double-booking prevention, parameterized search filtering, and 12 SQL-driven business intelligence analytics, fully containerized via Docker Compose.

1. Problem Statement & Background

Hotel enterprises operate interconnected domains including room inventory, front-desk reservations, customer contacts, culinary dining, guest amenities, human resources, and housekeeping inspections. Manual or spreadsheet-based tracking causes grave operational bottlenecks: data redundancy, lack of referential constraints, conflicting overlapping reservations for the same room, and failure to distinguish booked revenue from actual cleared funds. This project resolves these issues with an enterprise relational database system.

2. Mandatory Technology Stack

Tier	Technology	Version / Specification	Role in Project
Database	MySQL Server	8.0 (InnoDB)	ACID transactions, CHECK & FK constraints
Backend	Python Flask	3.1.3	REST routing, input validation & transactions
Driver	PyMySQL	1.2.3	Pure-Python driver with DictCursor
Frontend	HTML5 / CSS3	Modern Standard	Deep navy corporate sidebar ERP layout
Templating	Jinja2	3.1.6	Server-rendered dynamic templates
Charts	Chart.js	4.4.1	Occupancy & monthly revenue visualizer
Container	Docker Compose	Compose v2+	Multi-service health orchestration

3. Relational Database Schema & Entities (3NF)

The database **hotel_management** contains 10 normalized relational entities containing 114 deterministic seed records:

Entity	Primary Key	Key Foreign Keys & Constraints	Sample Records
HOTEL	hotel_id	CHECK (star_rating BETWEEN 1.0 AND 5.0)	3 Properties
ROOM	room_id	FK → HOTEL, UNIQUE (hotel_id, room_number), CHECK status	20 Rooms
CUSTOMER	customer_id	UNIQUE (email), NOT NULL fields	15 Guests
EMPLOYEE	employee_id	FK → HOTEL, CHECK (salary > 0)	10 Staff
BOOKING	booking_id	FK → CUSTOMER, FK → ROOM, CHECK (dates, amount)	20 Bookings
PAYMENT	payment_id	FK → BOOKING, CHECK (amount > 0, method, status)	20 Payments
RESTAURANT	restaurant_id	FK → HOTEL, cuisine_type, opening_hours	3 Outlets
FOOD	food_id	FK → RESTAURANT, CHECK (category, price > 0)	15 Menu Items
SERVICE	service_id	FK → HOTEL, CHECK (price >= 0)	8 Amenities
HOUSEKEEPING	housekeeping_id	FK → EMPLOYEE, FK → ROOM, CHECK (status)	15 Logs

4. Core SQL Queries Demonstrating DBMS Concepts

The system implements parameterized queries across all modules. Below are three representative formulations:

A. Concurrency Double-Booking Overlap Check (Server-Side Validation)

```
SELECT COUNT(*) AS overlap_count
FROM BOOKING
WHERE room_id = %s
AND booking_status IN ('Confirmed', 'Completed')
AND check_in_date < %s
AND check_out_date > %s;
```

B. Monthly Revenue Trend (Only Completed Invoices via DATE_FORMAT)

```
SELECT
DATE_FORMAT(payment_date, '%Y-%m') AS payment_month,
COUNT(*) AS completed_transactions,
COALESCE(SUM(amount), 0) AS total_revenue_collected
FROM PAYMENT
WHERE payment_status = 'Completed'
GROUP BY DATE_FORMAT(payment_date, '%Y-%m')
ORDER BY payment_month ASC;
```

C. Room Occupancy Utilization Rate Formulation

```
SELECT
COUNT(*) AS total_rooms,
SUM(CASE WHEN room_status = 'Occupied' THEN 1 ELSE 0 END) AS occupied_rooms,
ROUND((SUM(CASE WHEN room_status = 'Occupied' THEN 1 ELSE 0 END) / COUNT(*)) * 100, 2) AS
occupancy_percentage
FROM ROOM;
```

5. Testing & Quality Assurance Verification

The application contains an automated test suite executed with **pytest**, achieving 100% pass rate:

Test Module	Test Case	Scope Verified	Outcome
test_database.py	test_config_defaults	Verifies default DB name, port 3306, user	**PASS**
test_database.py	test_init_sql_schema_entities	Verifies all 10 entity tables exist in init.sql	**PASS**
test_database.py	test_init_sql_constraints	Validates CHECK, FK, UNIQUE, NOT NULL	**PASS**
test_database.py	test_init_sql_seed_data	Confirms deterministic seed data counts	**PASS**
test_database.py	test_offline_handling	Tests graceful handling when DB offline	**PASS**
test_routes.py	test_health_endpoint	Verifies /health endpoint returns JSON status	**PASS**
test_routes.py	test_404_error_page	Verifies custom 404 template rendering	**PASS**
test_routes.py	test_dashboard_rendering	Verifies KPI subqueries & joined reservations	**PASS**
test_routes.py	test_rooms_filtering	Verifies multi-parameter filters & sorting	**PASS**
test_routes.py	test_customers_directory	Verifies customer search & spend sums	**PASS**
test_routes.py	test_payments_ledger	Verifies payments ledger & method breakdown	**PASS**
test_routes.py	test_booking_validation	Validates rejection of checkout <= checkin	**PASS**

6. Conclusion & Evaluation Readiness

The Grand Luxe Hotel Management ERP application successfully meets all mandatory academic criteria of the CAM 5 assignment. By combining clean database normalization, direct parameterized PyMySQL execution, responsive HTML5/CSS3 interface design, and containerized Docker Compose orchestration, the project demonstrates an end-to-end, enterprise-ready database application suitable for academic demonstration and faculty viva examination.